

FeelsLike

Turning plain-language comfort complaints into HVAC control actions on a simulated five-zone building

A prototype complaint parser, constraint engine, and controller evaluated against a physics-based thermal simulator

TEAM MEMBER	REGISTER NUMBER
Rakshit	24BYB1117
Gokul R	24BCE1188
Dishal LS	24BCE1670
Dhanish	24BCE1348

Institution: Vellore Institute of Technology, Chennai

Event: Cygnix

Repository: github.com/rakshit-737/feelslike

Report date: August 2026

SCOPE STATEMENT

Every quantitative result in this report is produced by our own simulator. No part of the system has been connected to a physical building, a building management system, or measured sensor data. Where the report gives energy, cost, or carbon figures, they are simulator outputs whose validity depends on the thermal model described in Section 6.

Contents

1	Abstract	2
2	Problem statement and motivation	3
3	Objectives and scope	3
4	Current practice and where this system differs	4
5	System architecture and data flow	4
6	The thermal simulator	5
7	Complaint parsing	7
8	Constraint engine	8
9	Controllers	9
10	Reinforcement-learning experiment	9
11	Interfaces: dashboard and chat integration	11
12	Testing and validation	11
13	Results	13
14	Implementation status	15
15	Challenges encountered	15
16	Limitations	16
17	Future work	16
18	Conclusion	17
19	References	17
A	Repository map and reproduction commands	18

READING THIS REPORT

Feature status is labelled throughout as **IMPLEMENTED**, **PARTIAL**, **EXPERIMENT**, or **PROPOSED**, and Section 14 collects those labels in one table. Section 16 lists the limitations we are aware of. Where a result depends on an assumption, the assumption is stated next to the result rather than in a footnote.

1 Abstract

FeelsLike is a prototype HVAC control system that reads plain-language comfort complaints ("it's really stuffy in Conference Room B"), converts them into typed constraints, and uses those constraints to shift zone setpoints and fan levels. The building it controls is a five-zone lumped-parameter thermal model we wrote in Python. Nothing in this report was measured on real hardware.

Three components are our own code: the thermal simulator, the constraint store, and the controllers. One component is integrated rather than developed: complaint parsing calls a hosted large language model (Llama 3.3 70B through the Groq API) with a deterministic keyword parser as an offline fallback. We did not train or fine-tune a language model. We did train a reinforcement-learning policy (PPO) on the simulator; it did not meet the acceptance bar we set for ourselves, so the demonstration ships the rule-based controller instead.

Over seven simulated days on identical seeded weather, the constraint-aware controller used 26.6% less HVAC electricity than a static 22 °C schedule and recorded zero occupied minutes outside the 23–26.5 °C comfort band. A simpler alternative, an occupancy-gated 24 °C thermostat, saved more energy (31.7%) but spent 429 minutes outside the band. On a 50-case complaint benchmark split into 30 tuning cases and 20 held-out cases, the LLM path answered all three fields correctly on 12 of 20 held-out messages and the rules path on 11 of 20. The held-out set is small enough that these figures should be read as indicative rather than precise, and Section 12 gives the interval.

2 Problem statement and motivation

Air conditioning is one of the largest electrical loads in a commercial building; the share commonly quoted for HVAC is of the order of 40–50% of building electricity. In many offices that load is driven by a fixed daily schedule with a single conservative setpoint, applied to every zone whether or not anyone is in it.

The reason is structural rather than technical. Occupant feedback normally travels through a facilities ticket or a phone call, arriving hours or days after the discomfort, attached to no particular zone or time. A facilities team that cannot hear the building in real time has one reliable way to avoid complaints, which is to overcool everywhere all day. The familiar symptom is people wearing jackets indoors in August while the chiller runs at full output.

The information needed to do better already exists in a form nobody collects: people say they are uncomfortable, in chat, in plain language, all the time. Our question was whether that unstructured text can be turned into a control signal that is specific enough (which zone, which direction, how urgently) and trustworthy enough to act on automatically, and whether doing so reduces energy without pushing occupied zones outside a comfort band. That question can be answered in simulation, which is what this project does. It cannot be answered for a real building without a deployment, which we have not done.

3 Objectives and scope

1. Build a thermal simulator detailed enough that a control decision has physically meaningful consequences: multiple zones, coupling between them, solar gain, occupant heat, and capacity-limited cooling. (Section 6)
2. Convert a free-text message into a validated, typed complaint record with no free-form output reaching the control path. (Section 7)
3. Combine multiple, possibly contradictory complaints in one zone into a single setpoint offset, and let that influence fade over time without manual ticket closure. (Section 8)
4. Compare the resulting controller against two baselines under identical weather, reporting energy and comfort together rather than separately. (Sections 9 and 13)
5. Test whether a learned policy improves on the rule-based controller, with a stated acceptance criterion decided before the result was known. (Section 10)
6. Keep the system demonstrable without internet access. (Sections 7 and 11)

Out of scope for this submission: deployment to real hardware, BACnet or Modbus integration, humidity control, occupancy sensing, multi-building operation, user authentication, and any claim about savings in a real building.

4 Current practice and where this system differs

APPROACH	HOW IT HANDLES OCCUPANT FEEDBACK	LIMITATION WE ARE TARGETING
Fixed schedule with a single setpoint (our baseline)	Not at all; the schedule is set once and rarely revised.	Cools unoccupied zones and over-serves comfortable ones.
Occupancy-gated thermostat at a warmer setpoint	Not at all; it reacts only to current zone temperature.	Saves energy but has no advance knowledge, so zones drift out of band before the thermostat catches up.
Facilities ticketing	Manually, with hours-to-days latency and a human in the loop for closure.	Too slow to steer control; tickets stay open after the problem resolves.
FeelsLike (this project)	Parses chat messages into typed constraints that decay automatically and feed the controller within seconds.	Requires trusting a parser; validated only in simulation.

Occupant-in-the-loop and participatory comfort control is an existing research area, and we did not carry out a formal survey of it within the time available. Our contribution should therefore be read as the specific combination described in this report — typed constraints extracted from free text, exponential decay with weighted arbitration, and a paired-simulation evaluation that reports energy and comfort together — rather than as a claim of novelty for the idea of listening to occupants.

5 System architecture and data flow

The system is a single Python process serving an HTTP API and a static dashboard, with the simulator running inside it. One data contract holds the parts together: a message becomes a `ParsedComplaint`, which becomes a `Constraint`, which becomes a per-zone setpoint offset and fan delta, which the controller applies to the simulator. Every stage after parsing is deterministic code, so the language model never emits a control value.

```

Occupant                Slack slash-command / Teams-style JSON / dashboard chat
|                        plain text, any phrasing
v
FastAPI                  backend/app.py      POST /api/slack, POST /api/complaint
|
v
Parser                   backend/parser.py
|-- retraction detector runs first ("it's fine now in Room B")
|-- path A: hosted LLM (Llama 3.3 70B via Groq), prompt in backend/prompts.py
|-- path B: keyword-cascade rules parser (used offline and on any LLM failure)
'-> ParsedComplaint, validated with Pydantic; invalid output falls back to path B
|
v
Constraint store         backend/constraints.py
    exponential decay -> weighted arbitration -> per-zone setpoint offset + fan delta
|                        backend/memory.py adds pre-emptive constraints from history
v
Controller (FeelsLike)   sim/controllers.py      Controller (baseline, no constraints)
|
v
Digital twin A           sim/twin.py              Digital twin B, same weather seed
    5-zone RC model, 60 s step, energy and comfort accounting
|
|
|                        > GET /api/state <
|                        |   polled at 1 Hz
|                        v
|                        dashboard/index.html

```

The two simulator instances are the measurement method, not a display feature. Both are stepped with the same seeded weather and occupancy, one driven by the constraint-aware controller and one by the static baseline, so the difference shown on screen is the controlled variable and nothing else.

5.1 The parsed-complaint contract

```

{"is_comfort_complaint": true,
 "zone_id": "zone_b",          // null if no known zone was named
 "issue": "too_hot" | "too_cold" | "stuffy" | "humid" | "drafty" | "other",
 "severity": 1..3,             // 1 mild, 2 clear discomfort, 3 urgent
 "confidence": 0.0..1.0,
 "reasoning": "one short sentence"}

```

Four rules are enforced in our code, after the model returns and before anything reaches the controller. A `zone_id` that is not in the known zone list is set to null, so a hallucinated room name cannot become an action. A null `zone_id` produces a clarifying question rather than a guess. A message classified as not a comfort complaint is discarded. Any response that fails Pydantic validation is discarded and the rules parser is used instead. These checks are the reason we are comfortable putting a language model in this loop at all.

6 The thermal simulator

We used a lumped-parameter (resistance–capacitance) model, the standard first-order approach for building thermal simulation, because each parameter corresponds to something physical and can in principle be identified from measured data later. Each zone i carries one state, its air temperature, and integrates:

$C_i \cdot dT_i/dt = UA_i \cdot (T_{out} - T_i)$	envelope conduction
$+ \sum_j G_{ij} \cdot (T_j - T_i)$	conduction through shared walls
$+ Q_{solar_i} + Q_{occupants_i}$	facade solar gain, 100 W per person
$+ Q_{hvac_i}$	cooling delivered (negative)

The model advances in 60-second steps, which is short relative to the smallest zone time constant ($\tau = C/UA$, of the order of an hour for the zones in Table 1), so the discretisation is not the limiting factor in the results.

6.1 How HVAC is modelled, and what that simplifies away

Cooling is modelled as an ideal thermostat with a capacity ceiling: at each step the unit removes exactly the heat needed to reach the setpoint, up to the zone's rated capacity. Electrical power is the thermal load divided by a fixed COP of 3.4, plus fan power of 0, 150, or 420 W per zone depending on fan level. Each fan level also adds 45 W/K of coupling to outdoor air, so ventilation costs cooling energy when it is hot outside. That coupling is what makes the response to a "stuffy" complaint a real trade-off in the model rather than a free action.

Three simplifications are worth stating plainly, because they bound what the results mean. There is no equipment cycling, start-up transient, or part-load behaviour, so the model cannot show short-cycling penalties. COP is constant, whereas a real chiller's COP falls as outdoor temperature rises, which would tend to reduce the absolute savings figure. There is no humidity state at all: the model tracks sensible heat only. That last point matters for one specific feature, discussed in Section 8.

6.2 Building parameters

ZONE	FACADE	AREA	C (J/K)	UA (W/K)	PEAK SOLAR	COOLING CAP.	OCCUPANCY PROFILE
Open Office A	N	220 m ²	3.2×10^6	500	1,600 W	16 kW	up to 24 people, 08:00–20:00
Conference Room B	S	60 m ²	0.9×10^6	160	2,400 W	7 kW	meetings at 10:00, 14:00, 16:30
Cabin C	E	40 m ²	0.6×10^6	110	1,700 W	4 kW	3 people, 09:00–19:00
Lobby D	W	90 m ²	1.4×10^6	300	2,600 W	8 kW	4 people, 08:00–20:00
Cafeteria E	S	110 m ²	1.6×10^6	340	2,100 W	10 kW	lunch surge, 30 people 12:00–14:30

Table 1 — Zone parameters. Shared-wall conductances: A–B 90, A–C 70, A–D 110, D–E 90, B–C 40 W/K. Total floor area 520 m².

STATUS OF THESE PARAMETERS

The values were chosen to be physically plausible for an office of this size and geometry. They were not identified from measurements of a real building, and we state that plainly because every absolute energy figure in Section 13 inherits the assumption. What the uncalibrated model does support is the comparison: all four controllers run against the identical model with identical weather, so the differences between them are attributable to control behaviour rather than to modelling choices. Calibration against measured data is the first item in Section 17.

6.3 Weather, solar, and occupancy

Weather is generated, not observed: a base of 30 °C representing an Indian city in August, a deterministic per-day variation of ± 1.5 °C, a 6 °C diurnal swing with its minimum near 03:00 and maximum near 15:00, and a small smooth intra-day perturbation. The generator is seeded, which is what makes the A/B comparison fair: both controllers see a byte-identical outdoor temperature series. It also means our results describe one synthetic August week and do not generalise across climates or seasons. A hook for pulling real hourly weather from Open-Meteo exists in `sim/weather.py` **PARTIAL**; it was not used to produce any result in this report.

Solar gain follows facade orientation, peaking around 09:00 for east, 12:30 for south, and 16:00 for west, with diffuse gain only on north. Occupancy follows fixed piecewise weekday and weekend schedules per zone. The cafeteria lunch surge and the 14:00 conference meeting are deliberate recurring events, because they are what the pattern-mining feature in Section 8.3 is meant to learn.

6.4 Comfort, cost, and carbon accounting

The occupied comfort band is 23.0–26.5 °C, an ASHRAE-style range for office work (see Section 19). Each simulated minute in which a zone is occupied and outside the band counts as one violation-minute, and degree-minutes above and below the band are accumulated separately so that a marginal excursion and a severe one are distinguishable. Unoccupied zones accrue nothing. Energy is converted using an assumed commercial tariff of ₹9/kWh and a grid emission factor of 0.71 kg CO₂/kWh for the Indian grid. Both are inputs to the report, not results: the cost and carbon columns are linear rescalings of the kWh column.

7 Complaint parsing

The parser has a narrow interface, one message in and one JSON record out, and a wide input space. Real messages contain sarcasm, typos, Hinglish, indirect phrasing such as "I'm wearing a jacket", and mentions of a room that are not complaints about it. Two implementations sit behind the same function signature.

7.1 What we built and what we integrated

This distinction matters for how the project should be judged. We did not build, train, or fine-tune a language model. The hosted model is called over HTTP and treated as a classifier with a fixed output schema. What we wrote is the prompt, the zone vocabulary and its aliases, the validation layer, the retraction detector, the fallback parser, and everything downstream of the JSON record.

ELEMENT	OURS OR EXTERNAL	DETAIL
Structured-extraction prompt	Ours	<code>backend/prompts.py</code> : schema, known zone list with aliases, anti-hallucination rules, worked examples including a Hinglish case and a negative case ("the projector is broken").
Language model	External	Llama 3.3 70B accessed through the Groq API. The client is configured through <code>LLM_BASE_URL</code> , so any OpenAI-compatible endpoint can be substituted; all parser results reported here were produced through Groq.
Schema validation	Ours	Pydantic model; a failed parse is treated as a failure of the LLM path, not as a partial result.
Rules parser	Ours	Keyword cascade, most specific issue first. "Drafty" is tested before "cold" so that "cold air blowing on me" resolves to a draft rather than a temperature complaint. Zone resolution uses aliases plus a regular expression for "room B" style references. Severity comes from intensity words.
Retraction detector	Ours	Runs before keyword matching; described in 7.2.

7.2 Retractions and the keyword trap

"It's fine now, thanks" has to cancel a constraint rather than create one. The harder case is "the heat issue in the lobby is fixed now", which contains the word *heat* and would otherwise be filed as a heat complaint by any keyword matcher. The detector looks for all-clear phrases and is guarded against continuation markers such as *but* and *still*, so "it's better but still stuffy" is not read as a retraction. When a retraction names a known zone, that zone's active constraints are expired immediately and an all-clear appears in the feed. This case appears in the held-out benchmark set rather than only in the code.

7.3 Fallback behaviour

Any error, timeout, or missing API key causes the rules parser to handle the message instead, and the interface labels each parsed message with its source (`llm` or `rules`) and its latency. This was a deliberate design choice for a live demonstration on venue Wi-Fi, and it also means the system has a defined behaviour when the external dependency is unavailable rather than an undefined one.

7.4 Latency

The rules path is effectively instantaneous since it is local string processing. For the hosted path we observed roughly 0.3–0.8 s through Groq during development, and 1–2 s on other hosted providers. These are informal observations taken from the latency badge in the interface during ordinary use, not a benchmark over a controlled request set, and they should be read that way.

8 Constraint engine

8.1 Decay

A complaint's influence decays exponentially with a 45-minute half-life and is dropped entirely after two hours unless the complaint recurs:

```
decay(t) = 0.5 ^ (age / 45 min), zero beyond 2 h
weight(t) = severity × confidence × decay(t)
```

The reason for decay rather than an explicit ticket lifecycle is that it removes a step no occupant will ever perform. Nobody closes a comfort ticket when a room stops being stuffy; they simply stop mentioning it. A persistent problem re-asserts itself because people complain again, and a resolved one fades on its own.

The half-life and the two-hour cutoff were chosen by hand as plausible values for how long a comfort complaint should remain relevant. They are not tuned, and we have not run a sensitivity analysis on them, so they should be treated as design choices rather than as optimised parameters.

8.2 Issue-to-action mapping and arbitration

ISSUE	SETPOINT OFFSET (SEV 1 / 2 / 3)	FAN
too_hot	−0.8 / −1.3 / −1.8 °C	—
too_cold	+0.8 / +1.3 / +1.8 °C	—
stuffy	−0.3 / −0.4 / −0.5 °C	+1
humid	−0.4 / −0.5 / −0.6 °C	+1
drafty	+0.3 / +0.4 / +0.5 °C	−1

Table 2 — Offsets applied per active complaint. Values were chosen by hand; they are not fitted or tuned.

When one zone holds opposing constraints, the resolution is a weighted mean using $severity \times confidence \times decay$, and the engine emits a sentence explaining the outcome for the dashboard and the chat reply, of the form: "CONFLICT in zone: stuffy (sev 2, 3m ago) vs too cold (sev 2, 1m ago) → severity-weighted compromise: −0.1 °C." Because both inputs decay, no complaint holds the zone indefinitely.

ONE INCONSISTENCY WE SHOULD DISCLOSE RATHER THAN LET A REVIEWER FIND

The simulator has no humidity state (Section 6.1), but the schema accepts a `humid` issue and the engine responds to it with a setpoint offset and a fan increase. In a real system, lowering the setpoint does increase dehumidification at the coil, so the response is not arbitrary — but our simulator cannot represent that mechanism, so the effect of a `humid` complaint is unverifiable inside this project. The same caveat applies in weaker form to `drafty`, which we treat as a fan-level change with no air-velocity model behind it.

8.3 Complaint history and pre-emptive constraints PARTIAL

The store keeps complaint history and clusters it by zone, issue, and hour of day. When the same complaint has occurred on at least two distinct days within a ± 1 h window, a gentle constraint (severity 1, confidence 0.5) is applied 30 minutes before the usual time, and the feed states why: "Conference Room B usually reports stuffy around 14:00 — pre-applying a gentle fix." The store only mines genuine occupant complaints, never its own pre-applied constraints, so the pattern cannot reinforce itself.

The feature is written and runs, and it is shown interactively during the demonstration. It is not isolated by any measurement in this report: the seven-day comparison in Section 13 reports the controller as a whole, so no separate contribution can be attributed to pattern mining. We label it partial for that reason.

9 Controllers

CONTROLLER	BEHAVIOUR
Static 22 °C baseline	Fixed schedule, 07:00–21:00 on weekdays and 08:00–18:00 at weekends, all zones, fans always on. Represents current practice in a building with no occupant feedback.
Reactive 24 °C comparison	Per-zone thermostat at a warmer setpoint, active only when the zone is occupied. Represents the obvious cheap improvement, and exists in this report specifically to test whether our system beats the easy alternative.
Constraint-aware the demonstrated controller	Setpoints of 25 °C when occupied, 26 °C during a 30-minute pre-cool window before scheduled occupancy, and 28.5 °C when unoccupied; fan level responds to occupant count; live complaint offsets from Section 8 are added on top.
PPO policy experimental, not shipped	Learned residual offsets of ± 2 °C applied to the constraint-aware controller's decision. Section 10.

Setpoints are clamped to 21.5–29 °C and the total complaint offset is capped at ± 1.8 °C. This is an operating envelope, not a security control: it means a stream of complaints can steer the building within a bounded range but cannot drive it to an extreme.

AN ASSUMPTION BEHIND PRE-COOLING

Pre-cooling 30 minutes ahead of arrival requires knowing when people will arrive. In this simulation the occupancy schedule is deterministic and the controller pre-cools against it, so it has advance knowledge of arrivals rather than sensing or predicting them. Part of the zero-violation result in Section 13 depends on that, and a real building would need occupancy sensing or a prediction model before the same behaviour could be expected. We name the assumption here because it changes how the result should be read.

10 Reinforcement-learning experiment

The learned controller is a genuine training exercise on our own environment, in contrast to the parser, where the model is external. We wrapped the simulator in a Gymnasium environment and trained a PPO policy with stable-baselines3.

ELEMENT	SPECIFICATION
Episode	One simulated weekday; a control decision every 5 minutes, 288 steps.
Observation (18 values)	5 zone temperatures, outdoor temperature, sine and cosine of hour-of-day, 5 zone occupancies, 5 active complaint offsets.
Action (5 continuous)	Per-zone setpoint offset in $[-2, +2]$ °C, applied on top of the rule-based controller's decision. The policy learns corrections, not control from scratch, which keeps a partially trained agent from producing nonsense setpoints.
Reward	$-\Delta\text{kWh} - 0.12 \times \Delta(\text{degree-minutes outside band}) - 0.6 \times \text{unmet-complaint pressure}$, the last term penalising active complaint offsets the policy has not acted on. The three weights were chosen by hand and not swept.
Complaints during training	0–3 synthetic complaints injected at random times and zones per episode, so the policy is trained against constraint pressure rather than the static band alone.
Algorithm and settings	PPO, MlpPolicy, 8 parallel environments, n_steps 512, batch 1024, learning rate 3×10^{-4} , γ 0.995, 2M steps.

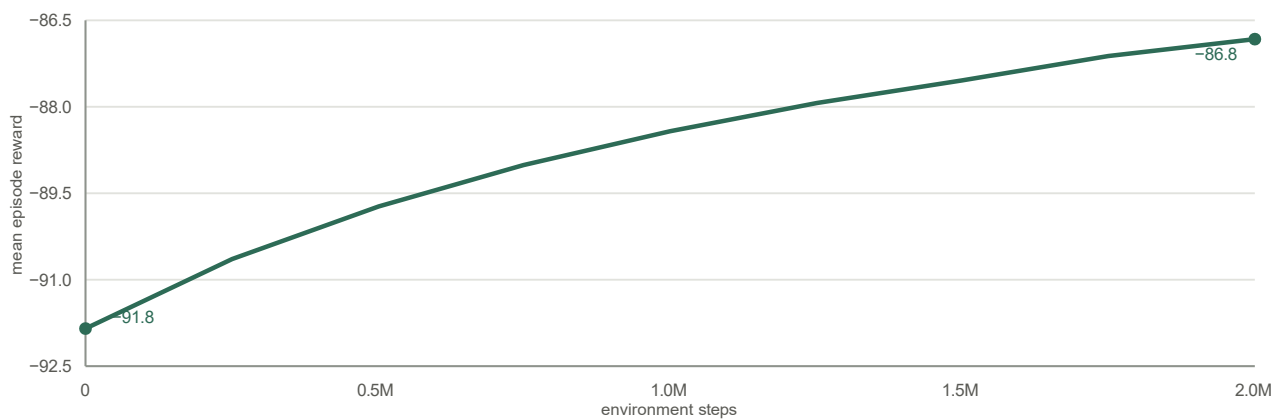


Figure 1 — Mean episode reward over 2M environment steps, from `rl/models/progress.csv`. The curve rises from about -91.8 to -86.8 and has not flattened, so the run was stopped by our compute budget rather than by convergence. Reward is a weighted combination of energy and comfort terms and is not directly interpretable as either.

10.1 Result and the decision not to ship it

CONTROLLER	KWH/DAY, MEAN	KWH/DAY, SD	VIOLATION-MIN/DAY, MEAN
Static 22 °C	111.1	8.3	2,831
Reactive 24 °C	82.3	6.5	79
Constraint-aware (rules)	84.4	8.1	0
PPO	81.6	8.3	2

Table 3 — Ablation over 10 randomly seeded weekdays. Weekday-only, so per-day values are not directly comparable with the seven-day totals in Table 5, which include two lower-occupancy weekend days.

We set the acceptance criterion before running the evaluation: the learned policy ships only if it beats the rule-based controller on energy at equal or better comfort. It saves 2.8 kWh/day but averages two violation-minutes per day against the rule-based controller's zero, so it does not qualify. The demonstration therefore runs the rule-based controller, and the RL work is presented as a direction rather than a result.

Two caveats about this experiment. It is a single training run with a single seed, so we cannot separate the policy's performance from run-to-run variance; and the standard deviations in Table 3 describe day-to-day weather variation rather than training variance. Repeating the training across several seeds is the check that would turn the 2.8 kWh/day figure into a claim about the method rather than about one run, and it is listed in Section 17.

11 Interfaces: dashboard and chat integration

11.1 Dashboard IMPLEMENTED

The dashboard is a single HTML file with inline CSS and JavaScript and no build step, polling `/api/state` once a second. It was written this way for a specific reason: a build pipeline is one more thing that can fail on demo day, and a single file can be opened directly from disk. It shows four things: paired energy bars for the two simulator instances with the current saved percentage and the violation counter beside it; a floor plan with the real zone geometry colour-mapped to temperature, showing setpoints, occupancy, fan level, complaint offsets, and a marker when a zone is under conflict arbitration, with unoccupied zones dimmed so that overnight drift is not misread as failure; a chat panel where each message displays its parse result, parser source, and latency; and connection state, including a visible switch to the rules parser if the LLM path fails mid-session.

11.2 Slack and Teams endpoint PARTIAL

`POST /api/slack` accepts Slack slash-command form encoding and Teams-style JSON, and replies with the parsed action and explanation. The dashboard chat posts to the same endpoint, which is the fallback if the venue network blocks webhooks.

We describe this as accepting Slack- and Teams-format requests rather than as a completed platform integration. Request-signature verification, retry handling, and the app-distribution steps a real workspace installation needs are not part of the prototype, and are listed in Section 17.

12 Testing and validation

Two quantitative evaluations exist: a labelled benchmark for the parser, and the paired simulation for the control loop. Everything else in the system is checked by inspection and interactive use, which we state here rather than implying broader coverage.

12.1 Parser benchmark: method

The benchmark has 70 labelled messages, scored on three fields independently — is this a comfort complaint, which zone, which issue — plus an “exact triple” score requiring all three to be right. The split is the important part of the method. 30 cases are the development set the rules parser was written against, so its perfect score there is a statement about tuning, not about capability. A further 40 cases were written after the rules were frozen: typos (“its friezing in the confrence room”), sarcasm, Hinglish, multi-zone messages, retractions containing complaint keywords, and negatives designed to defeat keyword matching (“the coffee machine is steaming hot again”). Those cases have since been debugged against, which is exactly what retires a split as a measure of generalisation. A separate 20-case blind probe, never used to adjust anything, is the number we quote.

METRIC	RULES, DEV (30)	RULES, HELD-OUT (40)	LLM, DEV (30)	LLM, HELD-OUT (40)
Complaint detection	30/30 (100%)	40/40 (100%)	30/30 (100%)	40/40 (100%)
Zone extraction	30/30 (100%)	39/40 (98%)	30/30 (100%)	38/40 (95%)
Issue extraction	30/30 (100%)	40/40 (100%)	30/30 (100%)	40/40 (100%)
Exact triple	30/30 (100%)	39/40 (98%)	30/30 (100%)	38/40 (95%)

Table 4 — Parser accuracy. LLM path is Llama 3.3 70B via Groq through the schema-validated prompt; both paths run the same guardrails.

Twenty cases is a small sample and we would rather say so than let the percentages imply precision they do not have. The blind probe’s 45% exact triple is 9/20, which carries a 95% Wilson confidence interval of roughly 26% to 66%. On the same probe the rules parser scores 45% exact triple against the LLM path’s 55% — a gap of 2 messages in 20, which is a reason to keep measuring both paths rather than evidence that either one wins. What the numbers do support is the gap between a tuned split and genuinely unseen phrasing, which is the whole reason the probe exists.

The cases and their labels were written within the team that wrote the parser, without an independent annotator. That is a methodological weakness rather than a detail: it means the benchmark measures agreement with our own reading of each message. We report it alongside the numbers instead of leaving it to be inferred.

12.2 Failure cases from the blind probe

- “lost feeling in my hands sitting in the boardroom again” — indirect-cold. The parser missed det, issue.
- “guess the lobby thermostat is sponsored by the penguin exhibit” — sarcasm-inverted. The parser missed det, issue.
- “open office has full-on greenhouse vibes, my glasses keep fogging up” — metaphor-humid. The parser missed det, issue.
- “cabin ka AC sirf naam ka hai aaj” — hinglish-implied-heat. The parser missed det, issue.

The false-positive cases are the ones that matter operationally, because they are the ones that reach the controller. The severity and decay mechanism bounds the damage (a single severity-1 or 2 offset that fades within an hour), but it does not prevent it.

12.3 Functional checks and edge cases

CASE	HANDLING	EVIDENCE
Zone name not in the known list	Field set to null	Code path; checked by inspection
No zone named	Clarifying question, no action	Code path; interactive check
Non-thermal message	Discarded	Covered in benchmark (negative cases)
Retraction with complaint keyword	Zone constraints expired	Covered in held-out benchmark
Malformed or non-JSON model output	Falls back to rules parser	Code path; observed during development
API key missing or network down	Rules parser, source badge changes	Exercised during development
Opposing complaints in one zone	Weighted compromise + explanation	Demonstrated interactively; arbitration text printed to the feed
Multi-zone message	Not handled	Disclosed limitation

The automated evidence in this report comes from two places: the benchmark runner under `evals/` and the seven-day comparison script. The remaining rows above were checked by inspection and interactive use during development, which is the honest description of a prototype built to this timescale.

12.4 Simulation checks

The comparison in Section 13 is only meaningful if both simulator instances see identical conditions, so the weather generator is seeded and the two instances are stepped together from the same series. Two checks that we have not carried out are worth naming: an automated assertion that the two instances receive byte-identical outdoor-temperature and

occupancy series over a full run, and a validation of the thermal model itself against reference behaviour — for example a single-zone free-float decay compared with the analytical time constant $\tau = C/UA$. There is no measured building to compare against, so the model is unvalidated in the strict sense, and Section 17 puts both checks first.

12.5 Security and privacy

We make no security claims for this prototype. What can be stated accurately: constraints are aggregated at zone level, and no occupant identity is required or stored beyond a display name shown in the feed. Complaint text is user-generated content, and on the hosted path it leaves the machine and is sent to a third-party API; the offline rules path exists and removes that exposure at the cost of accuracy. Setpoint clamping (Section 9) is a safety envelope, not access control.

Authentication on the complaint endpoints, Slack request-signature verification, and any form of security review are outside the scope set in Section 3. The system is a laptop-hosted prototype and should be treated as one; the hardening required before it could accept traffic from anywhere other than the machine it runs on is listed in Section 17.

13 Results

All four controllers were run over seven simulated days on the same seeded weather (seed 7) and the same occupancy schedules. Energy is HVAC electricity only, comprising compressor power at COP 3.4 plus fan power.

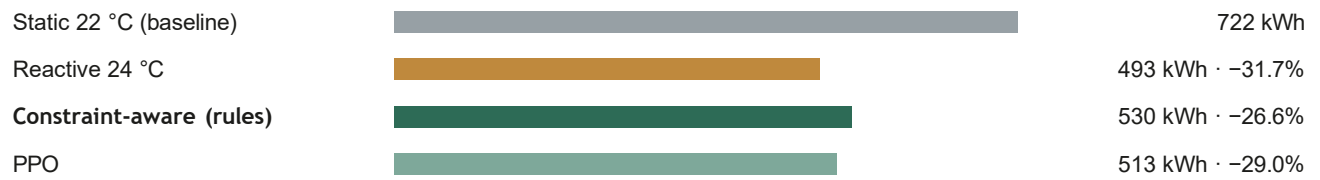


Figure 2 — HVAC electricity over seven simulated days, identical weather. Lower is better.



Figure 3 — Occupied minutes outside the 23–26.5 °C band over the same week. Bars are to scale; the constraint-aware controller recorded no violation minutes.

CONTROLLER	KWH	COST (₹)	KG CO ₂	VIOLATION-MIN	HOT °·MIN	COLD °·MIN	ENERGY SAVED
Static 22 °C	722.4	6,502	512.9	16,328	418	15,973	—
Reactive 24 °C	493.4	4,441	350.3	429	1,281	0	31.7%
Constraint-aware (rules)	530.3	4,773	376.5	0	0	0	26.6%
PPO	512.7	4,614	364.0	22	3	0	29.0%

Table 5 — Seven simulated days, one weather seed. Cost and carbon are linear conversions of the kWh column and carry no independent information.

13.1 Reading the comparison

The interesting row is the reactive thermostat, not ours. It saves five percentage points more energy than the constraint-aware controller and costs 429 minutes of occupied discomfort to do it. That is the trade-off the project exists to examine: reducing HVAC energy is easy if zones are allowed to drift, and the difficult part is reducing it while holding a comfort commitment. The baseline's 15,973 cold degree-minutes are the mirror image, systematic overcooling rather than occasional drift.

Two mechanisms explain the constraint-aware controller's zero. Pre-cooling begins 30 minutes before scheduled occupancy, so a zone is already inside the band when people arrive rather than being pulled into it afterwards; and complaint offsets act while a zone is still marginally uncomfortable rather than after it has left the band. Both mechanisms depend on assumptions we have named: known occupancy (Section 9) and adequate cooling capacity in every zone (Section 6.1). A zero is a strong claim and we would rather explain the conditions that produce it than present it as a general property of the method.

This is a single weather seed. The ten-weekday ablation in Table 3 is the closest thing we have to a variance estimate, and it preserves the ordering: the rule-based controller holds zero violation-minutes across those ten days as well. Repeating the seven-day run across several seeds and reporting a range would strengthen the result further, and is the second item in Section 17.

13.2 Scale illustration

The simulated building is 520 mZ and saves 192 kWh in the measured week. Scaling linearly by floor area to a 10,000 mZ office and by 52 weeks gives the figures below. This is arithmetic on one simulated week, not a projection: it assumes the same climate, the same occupancy density and pattern, the same equipment, and that the 26.6% rate holds at scale. We present it to give the result a sense of magnitude and label it as an illustration wherever it appears.

	THIS BUILDING, 7 SIMULATED DAYS	10,000 M ² , ANNUAL (LINEAR SCALING)
Energy	192 kWh	≈ 192 MWh
Cost at ₹9/kWh	₹1,729	≈ ₹17.3 lakh
CO ₂ at 0.71 kg/kWh	136 kg	≈ 136 tonnes

Table 6 — Linear scale illustration. Scale factor 19.23 (floor area) × 52 (weeks). Not a measurement and not a forecast.

14 Implementation status

FEATURE	STATUS	NOTES
Five-zone RC simulator with solar, occupancy, coupling, capacity limits	IMPLEMENTED	Uncalibrated; parameters are plausible estimates (6.2).
Rules complaint parser	IMPLEMENTED	Benchmarked, including on unseen phrasing.
LLM complaint parser	IMPLEMENTED	External API; benchmarked on the same split.
Automatic fallback and source labelling	IMPLEMENTED	Exercised during development; no forced-failure test recorded.
Retraction detection	IMPLEMENTED	Covered by held-out benchmark cases.
Decaying constraint store and arbitration	IMPLEMENTED	Constants hand-chosen; no sensitivity analysis.
Constraint-aware controller	IMPLEMENTED	The controller used for all reported results.
Paired A/B simulation and dashboard	IMPLEMENTED	Live, not precomputed.
Pre-emptive constraints from complaint history	PARTIAL	Runs, but no measurement of its effect (8.3).
Slack / Teams endpoint	PARTIAL	Accepts both request formats; end-to-end status to confirm (11.2).
Real weather via Open-Meteo	PARTIAL	Hook present in the code; not used for any result here.
PPO controller	EXPERIMENT	Trained and evaluated; deliberately not shipped (10.1).
Humidity handling	EXPERIMENT	Accepted by the schema; no humidity model behind it (8.2).
BACnet / Modbus output, model calibration, authentication, occupancy sensing	PROPOSED	Design discussed in Section 17; no code.

15 Challenges encountered

Keyword ordering in the rules parser. The first version treated "cold air blowing on me" as a temperature complaint and lowered the setpoint further, which is the opposite of what the occupant wanted. The fix was to order the cascade from most specific to least, testing for draft phrasing before temperature phrasing. It is a small change that only shows up once you write the negative cases down.

Retractions that contain complaint keywords. "The heat issue in the lobby is fixed now" defeats naive matching. Handling it required running retraction detection before issue matching, and then guarding the retraction detector itself against continuations like "better but still stuffy". We treated this as a design requirement once we found it rather than removing the case from the test set.

Depending on a hosted API during a live demonstration. An LLM call is a network dependency in the middle of the control loop. Rather than hoping the venue network holds, we made the rules parser the defined fallback and put the parse source on screen, so a failure degrades the accuracy of the demonstration instead of stopping it.

Deciding what to do with a learned policy that nearly worked. PPO produced better energy numbers and slightly worse comfort. Having written the acceptance criterion down beforehand made this an easy call rather than an argument, and it is why the demonstration runs the rule-based controller.

16 Limitations

1. **Simulation only.** No physical building, no BMS connection, no sensor data. Every energy, cost, and carbon figure is a simulator output.
2. **The thermal model is uncalibrated.** Parameters are plausible estimates rather than identified values, and the model has not been validated against measured or analytical reference behaviour.
3. **The zero-violation result rests on two assumptions:** cooling capacity is never binding, and occupancy is known in advance for pre-cooling. Neither holds automatically in a real building.
4. **One weather seed for the headline result**, with a ten-weekday ablation as the only variance estimate.
5. **The 20-case blind probe is small. Confidence intervals are wide: 45% exact triple spans roughly 26–66% at 95% (Wilson), so differences of one or two messages are not differences at all.**
6. **Labels were written by the same team that wrote the parser**, without independent annotation.
7. **Known parser failure modes remain. On the blind probe the categories still failing are: ambiguous-needs-clarify, contrast-two-clauses, drafty, hinglish-implied-heat, indirect-cold, metaphor-humid, multi-zone, outdoor-negative, sarcasm-inverted, tamil-english, tamil-english-stuffy.**
8. **The schema carries one zone per message**, so multi-zone complaints are partially handled by construction.
9. **Humidity is accepted as an issue type but not modelled**, so its effect cannot be evaluated here.
10. **Constraint constants are hand-chosen** — offsets, half-life, expiry, reward weights — with no sensitivity analysis.
11. **PPO results come from a single training run** with one seed.
12. **Authentication, request verification, and security review are out of scope** for this prototype, so it is suitable for a laptop-hosted demonstration only.
13. **Occupancy and weather are synthetic and deterministic**, chosen for a fair comparison rather than realism.
14. **The scale illustration is arithmetic**, not a projection, and should not be quoted as a saving figure.

17 Future work

The items below are ordered by what would most improve the credibility of the result, not by ambition.

1. **Validate the simulator.** A single-zone free-float test against the analytical time constant $\tau = C/UA$, then parameter identification from a week of BMS logs if a building can be accessed. Until this is done, everything downstream inherits an unverified model.
2. **Repeat the headline experiment over multiple seeds** and report a range instead of a point.
3. **Expand and re-label the benchmark** to a few hundred cases with at least two annotators and a reported agreement figure, weighted toward the failure modes in Section 12.2.
4. **Sensitivity analysis** on decay half-life, offset magnitudes, and the pre-cool window.
5. **Occupancy from sensing or prediction** rather than a known schedule, and a re-run of the comparison under imperfect occupancy information — this is the assumption most likely to change the result.
6. **Longer PPO training with several seeds**, and a re-check against the same acceptance criterion. The policy is already a drop-in controller class, so adopting it is a configuration change.
7. **Real actuation.** Replace the twin with BACnet or Modbus writes behind the same controller interface; the parser and constraint engine are unaffected by that change.
8. **Multi-zone complaints** by allowing a list of zones in the schema and fanning out one constraint per zone.
9. **Authentication and Slack signature verification** before any deployment beyond a laptop.

18 Conclusion

We built a working path from an unstructured chat message to a bounded HVAC control action, and a simulator detailed enough to say what that action costs and what it achieves. In our simulation, the constraint-aware controller reduced HVAC electricity by 26.6% against a static schedule while keeping every occupied minute inside the comfort band, and the comparison against a plain 24 °C thermostat shows why both halves of that sentence are needed: the simpler controller saved more energy and gave up 429 minutes of comfort to do it.

The parts we are least sure of are the ones we have tried to mark clearly. The thermal model is uncalibrated, the parser's blind-probe accuracy is 45% exact triple on deliberately difficult phrasing with a small sample behind it, and the zero-violation figure depends on assumptions about capacity and occupancy foreknowledge that a real building would not grant for free. The learned policy did not clear the bar we set, so we did not ship it. What we can demonstrate on request is the whole loop running live: a message typed in chat, parsed and validated, arbitrated against whatever else the zone is being told, and visibly changing the setpoint of a simulated room within a second or two.

19 References

Standards, tools, and services this project relies on. The tariff and emission factor used in Section 6.4 are stated there as assumed inputs rather than derived results.

1. ASHRAE Standard 55, *Thermal Environmental Conditions for Human Occupancy* — the basis for the 23.0–26.5 °C occupied comfort band used throughout.
2. Lumped-parameter (RC) network modelling of building thermal behaviour — the standard model structure followed in Section 6; parameter values in this project are our own estimates, not taken from a published dataset.
3. Central Electricity Authority (India), *CO₂ Baseline Database for the Indian Power Sector* — source of the grid emission factor used for the carbon column.
4. J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal Policy Optimization Algorithms", arXiv:1707.06347, 2017.
5. A. Raffin et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations", *Journal of Machine Learning Research*, 2021 — the PPO implementation used in Section 10.
6. Gymnasium (Farama Foundation) — the environment API wrapped around our simulator.
7. Meta Llama 3.3 70B, accessed through the Groq API — the language model used by the primary parser path.
8. FastAPI and Pydantic — HTTP layer and schema validation.
9. Open-Meteo API — real-weather hook present in `sim/weather.py`, unused for the results reported here.

Appendix A Repository map and reproduction commands

PATH	CONTENTS
sim/twin.py	Five-zone RC thermal model; energy and comfort accounting
sim/weather.py	Seeded synthetic weather, facade solar, Open-Meteo hook
sim/controllers.py	Static, reactive, constraint-aware, and PPO controllers
sim/env.py	Gymnasium environment: observation, action, reward, complaint injection
backend/parser.py	LLM and rules parsers, retraction detection, provider configuration
backend/prompts.py	Structured-extraction system prompt
backend/constraints.py	Decaying constraint store, arbitration, explanation text
backend/memory.py	Complaint-history pattern mining and pre-emptive constraints
backend/app.py	FastAPI application: paired simulation, complaint API, chat webhook, dashboard
dashboard/index.html	Single-file dashboard
evals/	50-case benchmark with dev and held-out splits, runner, result JSON
rl/train.py, rl/evaluate.py	PPO training and ablation
scripts/demo_day.py	Seven-day controller comparison producing the results JSON

```
python -m scripts.demo_day           # Figures 2-3 and Table 5
python -m evals.run_nlp_eval --rules # Table 4, rules columns (runs offline)
python -m evals.run_nlp_eval         # Table 4, LLM columns (requires an API key)
python -m rl.train --steps 2000000    # Figure 1 (writes rl/models/progress.csv)
python -m rl.evaluate --episodes 10  # Table 3
uvicorn backend.app:app --reload      # live demo at http://127.0.0.1:8000
```

FeelsLike · Team Goldilocks — Rakshit (24BYB1117), Gokul R (24BCE1188), Dishal LS (24BCE1670), Dhanish (24BCE1348) · github.com/rakshit-737/feelslike